

Переповнення буфера с gdb

Кувавіна Софія

Переповнення буфера

1.1. В ОС Linux створіть файл overflow1.c такого змісту:

```
#include <string.h>
main (int argc, char *argv[]) {
char str1[10];
strcpy (str1, argv[1]);
}
```

та скомпілюйте виконуваний файл:

```
#gcc -ggdb -mpreferred-stack-boundary=2 -o overflow1 ./overflow1.c
```

У даній елементарній програмі наявна вразливість переповнення стека в процедурі strcpy.

Введіть команду:

```
[root@localhost hack]# export LANG=C
```

Створюю файл

```
(kali㉿kali)-[~]
$ vim overflow1.c
```

Вношу зміст

```
File Actions Edit View Help
#include <string.h>
main (int argc, char *argv[]) {
char str1[10];
strcpy (str1, argv[1]);
~
~
```

Компілюю файл

```
(kali㉿kali)-[~]
$ gcc -ggdb -mpreferred-stack-boundary=4 -o overflow1 overflow1.c
```

Запускаю програму з аргументом меншої довжини, ніж 8 символів

Бачимо результат

```
(kali㉿kali)-[~]
$ ./overflow1 ABC

Buffer content: ABC
```

Запускаю програму з аргументом довжини рівно 8 символів

Теж коректно

```
(kali㉿kali)-[~]
$ ./overflow1 ABCDEFGH

Buffer content: ABCDEFGH
```

Запускаю програму з аргументом довжини рівно 8 символів
В програмі переповнений буфер

```
(kali㉿kali)-[~]  
$ ./overflow1 AAAAAAAAAA  
  
Buffer content: AAAAAAAAAA  
zsh: segmentation fault ./overflow1 AAAAAAAAAA
```

1.2 Запустіть відладчик зі скомпільованою програмою і встановіть точки зупинки до і після виклику strcpy:

```
[root@localhost hack]# gdb -q overflow1
```

```
(gdb) list 1
```

```
1 #include <string.h>
```

```
2 main (int argc, char *argv[]) {
```

```
3 char str1[10];
```

```
4 strcpy (str1, argv[1]);
```

```
5 }
```

```
(gdb) b 4
```

```
Breakpoint 1 at 0x804832e: file overflow1.c, line 4.
```

```
(gdb) b 5
```

```
Breakpoint 2 at 0x8048342: file overflow1.c, line 5.
```

```
(gdb)
```

Відладчик дозволяє контролювати стан регістрів і пам'яті. Команда виведення вмісту комірок пам'яті має вигляд `x/[n]T[size] <адреса пам'яті>`, де:

`x` – команда;

`n` – кількість слів;

`T` – формат представлення (`x` – hex, `s` – string, `i` – instruction, `d` – decimal, `c` – char, `b` – byte, `w` – word, `o` – octal, `u` – unsigned decimal, `t` – binary, `f` – float);

`[size]` – розмір слова (`b` – byte, `h` – halfword, `w` – word, `g` – giant);

`<адреса пам'яті>` може бути задана явно, через показчик або регістр.

Виведення значень регістрів: `i r <список регістрів>`.